Cloud Computing

V Sahiti Yellanki

Department of CSE- AIML & IoT

VNR VJIET

UNIT-II:

Virtualization: Introduction, Characteristics of virtualized environments: Increased security, Managed execution, Portability, Taxonomy of virtualization techniques: Execution virtualization, other types of virtualizations, Virtualization and cloud computing, Pros and cons of virtualization: Advantages of virtualization, The other side of the coin: disadvantages, Technology examples: Xen: para virtualization, VMware: full virtualization, Microsoft Hyper-V.

WHAT IS VIRTUALIZATION?

- Virtualization technology is one of the fundamental components of cloud computing, especially in regard to **infrastructure-based services.**
- Virtualization allows the creation of a secure, customizable, and isolated execution environment for running applications, even if they are untrusted, without affecting other users' applications.
- The basis of this technology is the ability of a computer program—or a combination of software and hardware—to emulate an executing environment separate from the one that hosts such programs.
- For example, we can run Windows OS on top of a virtual machine, which itself is running on Linux OS.

Introduction

Virtualization is a large umbrella of technologies and concepts that are meant to provide an abstract environment—whether virtual hardware or an operating system—to run applications.

The term virtualization is often synonymous with hardware virtualization, which plays a fundamental role in efficiently delivering Infrastructure-as-a-Service (IaaS) solutions for cloud computing.

virtualization technologies have a long trail in the history of computer science and have been available in many flavours by providing virtual environments at the operating system level, the programming language level, and the application level.

Moreover, virtualization technologies provide a virtual environment for not only executing applications but also for **storage, memory, and networking.**

Virtualization technologies have gained renewed interested recently due to the confluence of several phenomena:

- Increased performance and computing capacity.
- Underutilized hardware and software resources.
- Lack of space.
- Greening initiatives.
- Rise of administrative costs.

Characteristics of virtualized environments:

- Virtualization is a broad concept that refers to the creation of a virtual version of something, whether hardware, a software environment, storage, or a network.
- In a virtualized environment there are three major components: **guest, host, and virtualization** layer. The guest represents the system component that interacts with the virtualization layer rather than with the host, as would normally happen.
- The host represents the original environment where the guest is supposed to be managed.
- The virtualization layer is responsible for recreating the same or a different environment where the guest will operate (see Figure 3.1).

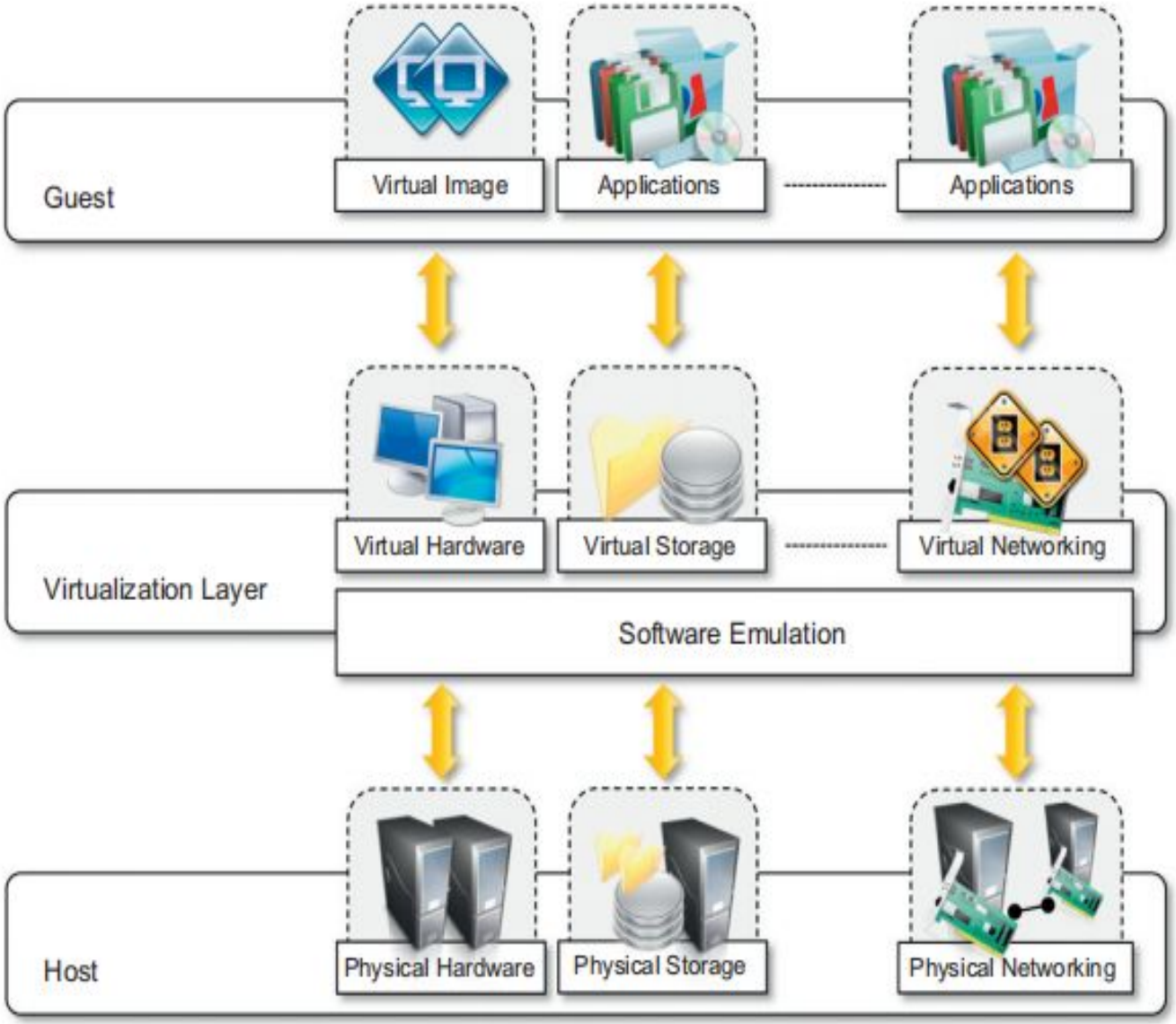


FIGURE 3.1

The virtualization reference model.

1 Increased security:

- The ability to control the execution of a guest in a completely transparent manner opens new possibilities for delivering a secure, controlled execution environment.
- The virtual machine represents an emulated environment in which the guest is executed. All the operations of the guest are generally performed against the virtual machine, which then translates and applies them to the host.
- This level of indirection allows the virtual machine manager to control and filter the activity of the guest, thus preventing some harmful operations from being performed.
- Resources exposed by the host can then be hidden or simply protected from the guest. Moreover, sensitive that is contained in the host can be naturally hidden without the need to install complex security policies. Increased security is a requirement when dealing with untrusted code.

2 Managed execution:

Virtualization of the execution environment not only allows increased security, but a wider range of features also can be implemented. In particular, **sharing, aggregation, emulation, and isolation** are the most relevant features (see Figure 3.2).

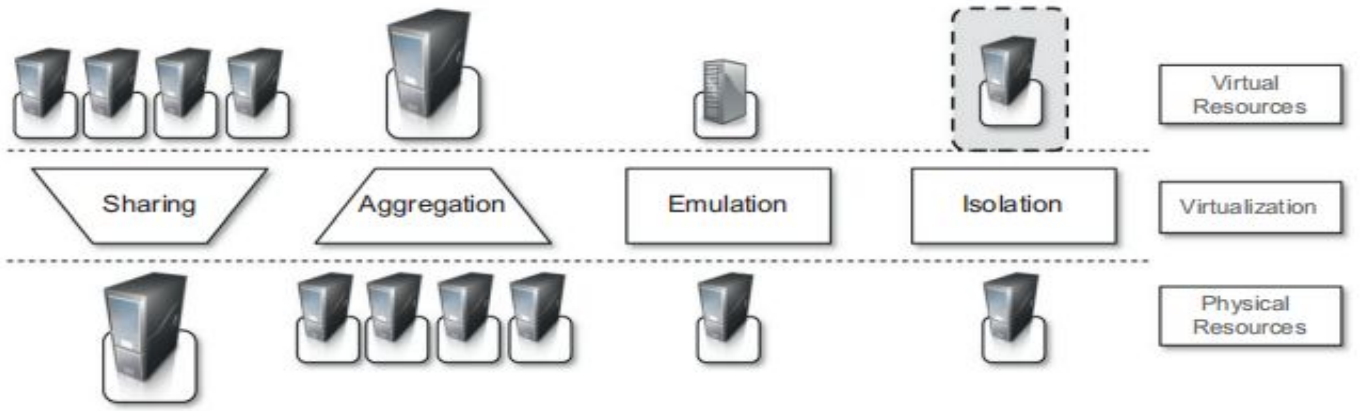


FIGURE 3.2

Functions enabled by managed execution.

- **Sharing:** Virtualization allows the creation of a separate computing environments within the same host. In this way it is possible to fully exploit the capabilities of a powerful guest, which would otherwise be underutilized. As we will see in later chapters, sharing is a particularly important feature in virtualized data centers, where this basic feature is used to reduce the number of active servers and limit power consumption.

- **Aggregation:** Not only is it possible to share physical resource among several guests, but virtualization also allows aggregation, which is the opposite process. A group of separate hosts can be tied together and represented to guests as a single virtual host. This function is naturally implemented in middleware for distributed computing, with a classical example represented by cluster management software, which harnesses the physical resources of a homogeneous group of machines and represents them as a single resource.
- **Emulation:** Guest programs are executed within an environment that is controlled by the virtualization layer, which ultimately is a program. This allows for controlling and tuning the environment that is exposed to guests. For instance, a completely different environment with respect to the host can be emulated, thus allowing the execution of guest programs requiring specific characteristics that are not present in the physical host. This feature becomes very useful for testing purposes, where a specific guest has to be validated against different platforms or architectures and the wide range of options is not easily accessible during development.

Isolation: Virtualization allows providing guests—whether they are operating systems, applications, or other entities—with a completely separate environment, in which they are executed. The guest program performs its activity by interacting with an abstraction layer, which provides access to the underlying resources. Isolation brings several benefits; for example, it allows multiple guests to run on the same host without interfering with each other. Second, it provides a separation between the host and the guest. The virtual machine can filter
the activity of the guest and prevent harmful operations against the host.

3 Portability:

The concept of portability applies in different ways according to the specific type of virtualization considered. In the case of a hardware virtualization solution, the guest is packaged into a virtual image
that, in most cases, can be safely moved and executed on top of different virtual machines. Except for the file size, this happens with the same simplicity with which we can display a picture image in different computers. Virtual images are generally proprietary formats that require a specific virtual machine manager to be executed. In the case of programming-level virtualization, as implemented by the JVM
or the .NET runtime, the binary code representing application components (jars or assemblies) can be run without any recompilation on any implementation of the corresponding virtual machine.

3.3 Taxonomy of virtualization techniques

Virtualization covers a wide range of emulation techniques that are applied to different areas of computing. A classification of these techniques helps us better understand their characteristics and use (see Figure 3.3).

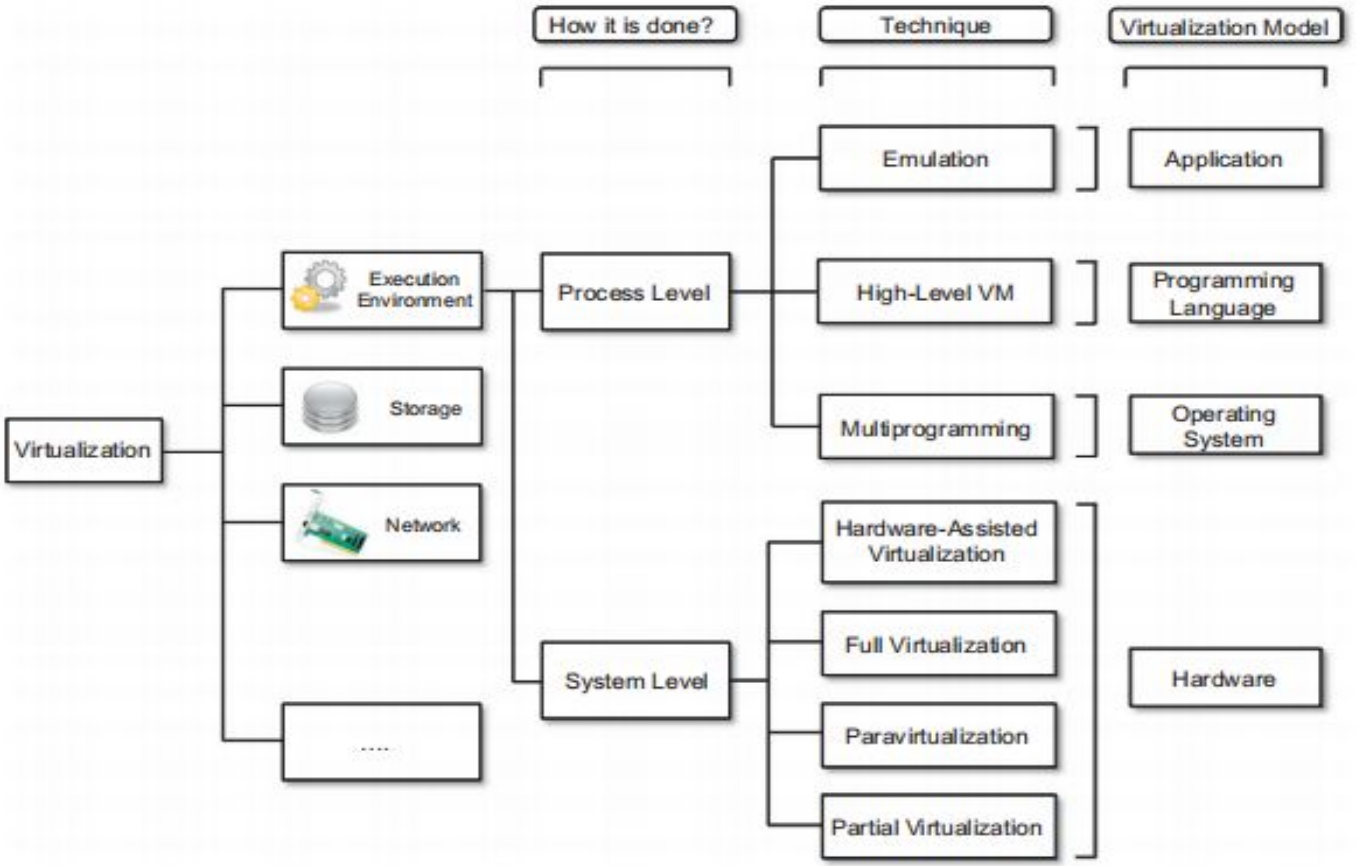


FIGURE 3.3

A taxonomy of virtualization techniques.

Virtualization is mainly used to emulate execution environments, storage, and networks. Among these categories, execution virtualization constitutes the oldest, most popular, and most developed area.

3.3.1 Execution virtualization

Execution virtualization includes all techniques that aim to emulate an execution environment that is separate from the one hosting the virtualization layer. All these techniques concentrate their interest on providing support for the execution of programs, whether these are the operating system, a binary specification of a program compiled against an abstract machine model, or an application. Therefore, execution virtualization can be implemented directly on top of the hardware by the operating system, an application, or libraries dynamically or statically linked to an application image.

1.1Machine reference model

Virtualizing an execution environment at different levels of the computing stack requires a reference model that defines the interfaces between the levels of abstractions, which hide implementation details. From this perspective, virtualization techniques actually replace one of the layers and intercept the calls that are directed toward it. Therefore, a clear separation between layers simplifies their implementation, which only requires the emulation of the interfaces and a proper interaction with the underlying layer.

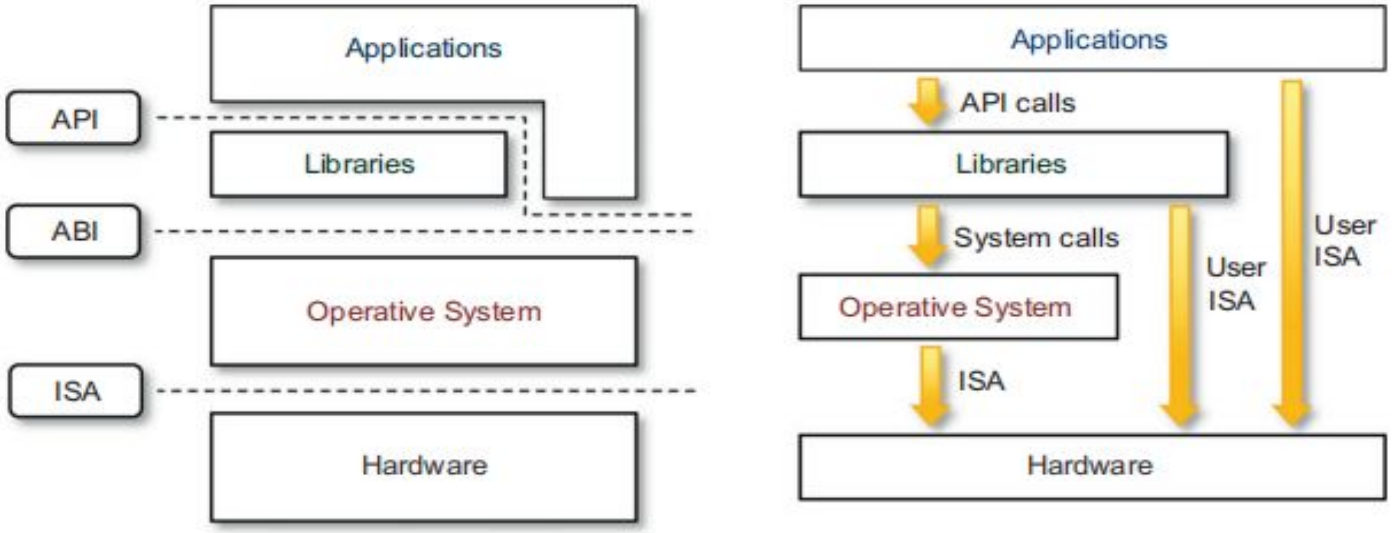


FIGURE 3.4

A machine reference model.

At the bottom layer, the model for the hardware is expressed in terms of the Instruction Set Architecture (ISA), which defines the instruction set for the processor, registers, memory, and interrupt management.

ISA is the interface between hardware and software, and it is important to the operating system (OS) developer (System ISA) and developers of applications that directly manage the underlying hardware (User ISA). The application binary interface (ABI) separates the operating system layer from the applications and libraries, which are managed by the OS. ABI covers details such as low-level data types, alignment, and call conventions and defines a format for executable programs. System calls are defined at this level. This interface allows portability of applications and libraries across operating systems that implement the same ABI. The highest level of abstraction is represented by the application programming interface (API), which interfaces applications to libraries and/or the underlying operating system.

For any operation to be performed in the application level API, ABI and ISA are responsible for making it happen. The high-level abstraction is converted into machine-level instructions to perform the actual operations supported by the processor. The machine-level resources, such as processor registers and main memory capacities, are used to perform the operation at the hardware level of the central processing unit (CPU). This layered approach simplifies the development and imple
mentation of computing systems and simplifies the implementation of multitasking and the coexistence of multiple executing environments.
The instruction set exposed by the hardware has been divided into different security classes that define who can operate with them. The first distinction can be made between privileged and nonprivileged instructions. Nonprivileged instructions are those instructions that can
be used without interfering with other tasks because they do not access shared resources. This category contains, for example, all the floating, fixed-point, and arithmetic instructions. Privileged instructions are those that are executed under specific restrictions and are mostly used for sensitive operations, which expose (behavior-sensitive) or modify (control-sensitive) the privileged state. For instance, behavior-sensitive instructions are those that operate on the I/O, whereas control-sensitive
instructions alter the state of the CPU registers. Some types of architecture feature more than one class of privileged instructions and implement a finer control of how these instructions can be accessed.

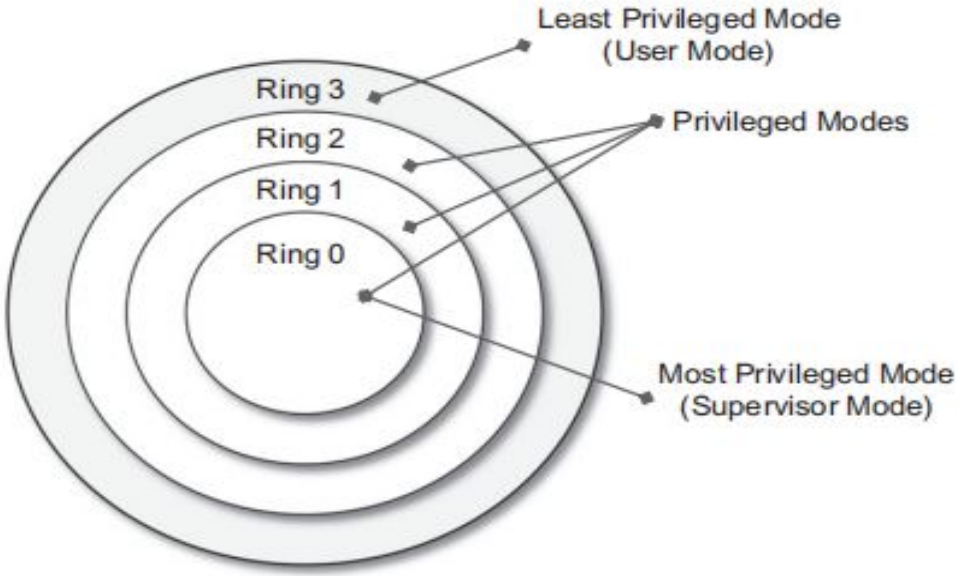


FIGURE 3.5

Security rings and privilege modes.

For instance, a possible implementation features a hierarchy of privileges (see Figure 3.5) in the form of ring-based security: Ring 0, Ring 1, Ring 2, and Ring 3; Ring 0 is in the most privileged level and Ring 3 in the least privileged level. Ring 0 is used by the kernel of the OS, rings 1 and 2 are used by the OS-level services, and Ring 3 is used by the user. Recent systems support only two levels, with Ring 0 for supervisor mode and Ring 3 for user mode.

All the current systems support at least two different execution modes: supervisor mode and user mode. The first mode denotes an execution mode in which all the instructions (privileged and non privileged) can be executed without any restriction. This mode, also called master mode or kernel mode, is generally used by the operating system (or the hypervisor) to perform sensitive operations on hardware level resources. In user mode, there are restrictions to control the machine-level resources. If code running in user mode invokes the privileged instructions, hardware interrupts occur and trap the potentially harmful execution of the instruction. Despite this, there might be some instructions that can be invoked as privileged instructions under some conditions and as non privileged instructions under other conditions.

The distinction between user and supervisor mode allows us to understand the role of the hypervisor and why it is called that. Conceptually, the hypervisor runs above the supervisor mode, and from here the prefix hyper- is used. In reality, hypervisors are run in supervisor mode, and the division between privileged and non privileged instructions has posed challenges in designing virtual machine managers.

It is expected that all the sensitive instructions will be executed in privileged mode, which requires supervisor mode in order to avoid traps. Without this assumption it is impossible to fully emulate and manage the status of the CPU for guest operating systems. Unfortunately, this is not true for the original ISA, which allows 17 sensitive instructions to be called in user mode. This prevents multiple operating systems managed by a single hypervisor to be isolated from each other, since they are able to access the privileged state of the processor and change it.

2 Hardware-level virtualization:

Hardware-level virtualization is a virtualization technique that provides an abstract execution environment in terms of computer hardware on top of which a guest operating system can be run. In this model, the guest is represented by the operating system, the host by the physical computer hardware, the virtual machine by its emulation, and the virtual machine manager by the hypervisor (see Figure 3.6).

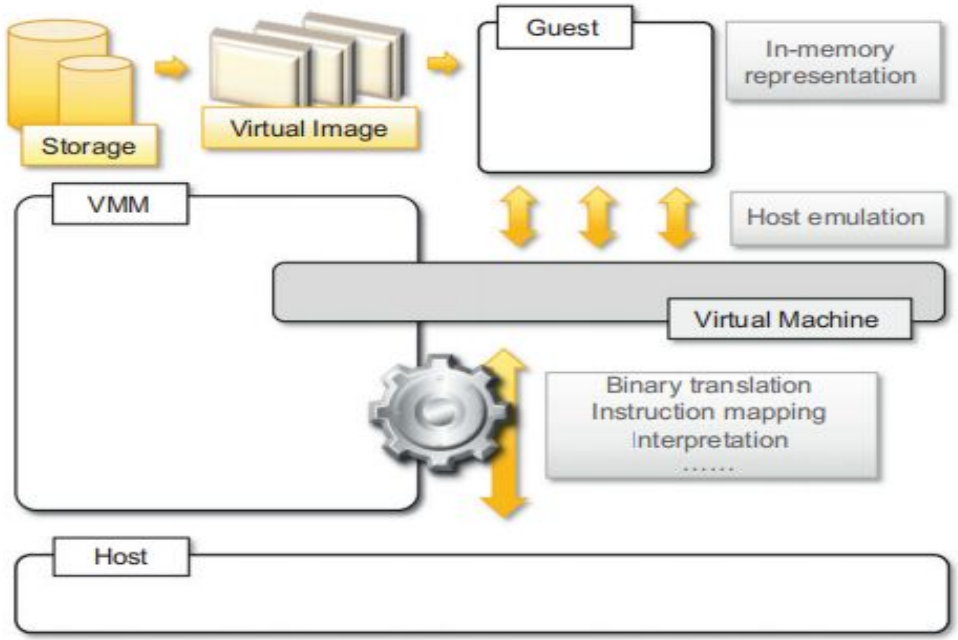


FIGURE 3.6

A hardware virtualization reference model.

The hypervisor is generally a program or a combination of software and hardware that allows the abstraction of the underlying physical hardware. Hardware-level virtualization is also called system virtualization, since it provides ISA to virtual machines, which is the representation of the hardware interface of a system. This is to differentiate it from process virtual machines, which expose ABI to virtual machines.

Hypervisors

A fundamental element of hardware virtualization is the hypervisor, or virtual machine manager (VMM). It recreates a hardware environment in which guest operating systems are installed. There are two major types of hypervisor: Type I and Type II (see Figure 3.7).

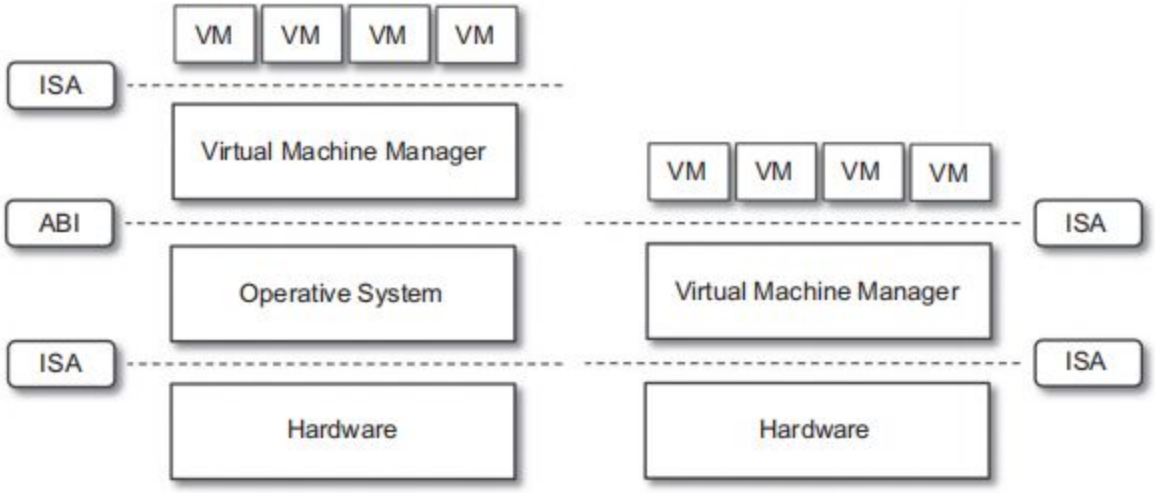


FIGURE 3.7

Hosted (left) and native (right) virtual machines. This figure provides a graphical representation of the two types of hypervisors.

Type I hypervisors run directly on top of the hardware. Therefore, they take the place of the operating systems and interact directly with the ISA interface exposed by the underlying hardware, and they emulate this interface in order to allow the management of guest operating systems. This type of hypervisor is also called a native virtual machine since it runs natively on hardware.

Type II hypervisors require the support of an operating system to provide virtualization services. This means that they are programs managed by the operating system, which interact with it through the ABI and emulate the ISA of virtual hardware for guest operating systems. This type of hypervisor is also called a hosted virtual machine since it is hosted within an operating system.

Conceptually, a virtual machine manager is internally organized as described in Figure 3.8.

Three main modules, dispatcher, allocator, and interpreter, coordinate their activity in order to emulate the underlying hardware. The dispatcher constitutes the entry point of the monitor and reroutes the instructions issued by the virtual machine instance to one of the two other modules.

The allocator is responsible for deciding the system resources to be provided to the VM: whenever a virtual machine tries to execute an instruction that results in changing the machine resources associated with that VM, the allocator is invoked by the dispatcher. The interpreter module consists of interpreter routines. These are executed whenever a virtual machine executes a privileged instruction: a trap is triggered and the corresponding routine is executed.

The design and architecture of a virtual machine manager, together with the underlying hardware design of the host machine, determine the full realization of hardware virtualization, where a guest operating system can be transparently executed on top of a VMM as though it were run on the underlying hardware.

.

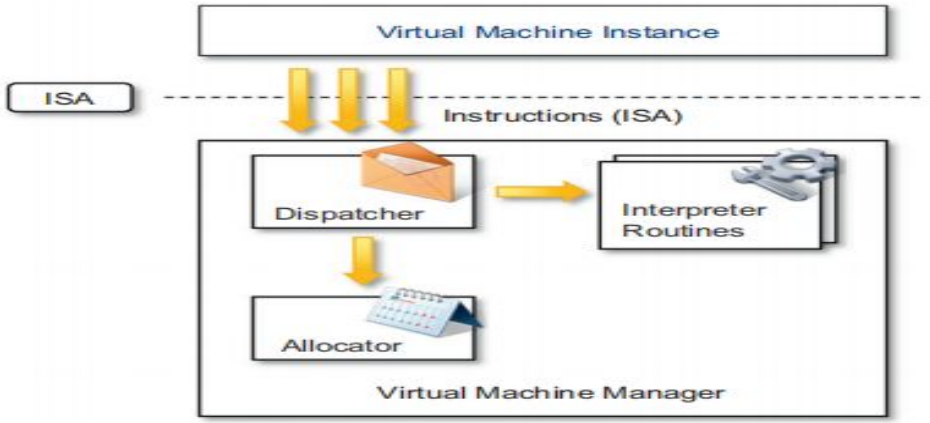


FIGURE 3.8
A hypervisor reference architecture.

Three properties have to be satisfied:

• Equivalence. A guest running under the control of a virtual machine manager should exhibit the same behaviour as when it is executed directly on the physical host.

• Resource control. The virtual machine manager should be in complete control of virtualized resources.

- Efficiency. A statistically dominant fraction of the machine instructions should be executed without intervention from the virtual machine manager.

The major factor that determines whether these properties are satisfied is represented by the layout of the ISA of the host running a virtual machine manager. Popek and Goldberg provided a classification of the instruction set and proposed three theorems that define the properties that hardware instructions need to satisfy in order to efficiently support virtualization.

THEOREM 3.1

For any conventional third-generation computer, a VMM may be constructed if the set of sensitive instructions for that computer is a subset of the set of privileged instructions.

This theorem establishes that all the instructions that change the configuration of the system resources should generate a trap in user mode and be executed under the control of the virtual machine manager. This allows hypervisors to efficiently control only those instructions that would reveal the presence of an abstraction layer while executing all the rest of the instructions without considerable performance loss. The theorem always guarantees the resource control property when the hypervisor is in the most privileged mode (Ring 0). The non privileged instructions must be executed without the intervention of the hypervisor. The equivalence property also holds good since the output of the code is the same in both cases because the code is not changed.

THEOREM 3.2

A conventional third-generation computer is recursively virtualizable if:

- It is virtualizable and
- A VMM without any timing dependencies can be constructed for it.

Recursive virtualization is the ability to run a virtual machine manager on top of another virtual machine manager. This allows nesting hypervisors as long as the capacity of the underlying resources can accommodate that. Virtualizable hardware is a prerequisite to recursive virtualization.

THEOREM 3.3

A hybrid VMM may be constructed for any conventional third-generation machine in which the set of user-sensitive instructions is a subset of the set of privileged instructions.

There is another term, hybrid virtual machine (HVM), which is less efficient than the virtual machine system. In the case of an HVM, more instructions are interpreted rather than being executed directly. All instructions in virtual supervisor mode are interpreted. Whenever there is an attempt to execute a behavior-sensitive or control-sensitive instruction, HVM controls the execution directly or gains the control via a trap. Here all sensitive instructions are caught by HVM that are simulated.

3. Hardware Virtualization Techniques:

(a)Hardware-assisted virtualization.: This term refers to a scenario in which the hardware provides architectural support for building a virtual machine manager able to run a guest operating system in complete isolation. This technique was originally introduced in the IBM System/370. At present,

examples of hardware-assisted virtualization are the extensions to the x86-64 bit architecture introduced with Intel VT (formerly known as Vanderpool) and AMD V (formerly known as Pacifica). These extensions, which differ between the two vendors, are meant to reduce the performance penalties experienced by emulating x86 hardware with hypervisors. Before the introduction of hardware-assisted virtualization, software emulation of x86 hardware was significantly costly from the performance point of view.

VMware Virtual Platform, introduced in 1999 by VMware, which pioneered the field of x86 virtualization, were based on this technique. After 2006, Intel and AMD introduced processor extensions, and a wide range of virtualization solutions took advantage of them: Kernel-based Virtual Machine (KVM), VirtualBox, Xen, VMware, Hyper-V, Sun XVM, Parallels, and others.

Hardware **virtualization** is a **technology that enables the creation of virtual instances of a physical computer system,** allowing multiple operating systems to run on a single physical machine.

(b) Full virtualization: Full virtualization is a virtualization technique that allows an operating system or application to run on a virtual machine **without any modification**, as if it were running directly on physical hardware.

- In full virtualization, the Virtual Machine Manager (VMM) or hypervisor provides a **complete emulation of the underlying hardware**.
- The guest operating system runs unmodified because it believes it is executing on real hardware.
- The hypervisor creates a fully isolated virtual environment for each virtual machine.
- One major advantage is **complete isolation**, which improves system security and fault tolerance.
- It enables the **coexistence of multiple operating systems** on the same physical platform.
- It also allows **emulation of different hardware architectures**, increasing flexibility.
- A key challenge in full virtualization is handling **privileged instructions** (such as I/O operations), since they directly affect hardware resources.
- These sensitive instructions must be intercepted and managed by the hypervisor to maintain control and isolation.
- Pure software-based virtualization can lead to **performance overhead** due to instruction trapping and emulation.

To improve efficiency, modern systems use **hardware-assisted virtualization**, where CPU extensions support virtualization and prevent harmful instructions from executing directly on the host. It creates virtual machines (VMs) that can run various operating systems, such as Windows, Linux, or others, without requiring any modifications to the guest OS.

(c)Paravirtualization: Para virtualization is a **non-transparent virtualization technique** that enables the implementation of thin and efficient Virtual Machine Managers (VMMs). Unlike full virtualization, para virtualization requires modification of the guest operating system to achieve better performance and simplified virtualization.

- In para virtualization, the hypervisor exposes a **modified software interface** to the virtual machine instead of providing a complete hardware abstraction.
- Para virtualization is a technique where the guest operating system is aware of the virtualization layer and cooperates with it.
- The guest operating system must be **modified and ported** to interact with this interface.
- Performance-critical operations (such as privileged instructions) are not fully emulated; instead, they are redirected to the hypervisor through special calls known as **hypercalls**.
- This approach reduces the overhead caused by instruction trapping and emulation, thereby **improving system performance**.
- The Virtual Machine Manager becomes simpler because it does not need to handle complex instruction emulation.
- Since OS modification is required, this technique is mainly suitable when the **source code of the operating system is available**, which is why it gained popularity in open-source and academic environments.

- Historically, similar concepts were used in IBM VM operating systems, but the term *para virtualization* was formally introduced in the **Denali project** at the University of Washington.
- A well-known implementation of para virtualization is the **Xen hypervisor**, which supports specially ported Linux operating systems.
- Operating systems that cannot be fully modified can still use **para virtualized device drivers** to improve performance.

Para virtualization improves virtualization efficiency by modifying the guest operating system to cooperate with the hypervisor. Although it requires OS modification, it offers better performance and simpler VMM implementation compared to full virtualization.

(d) Partial Virtualization: Partial virtualization is a virtualization technique in which only a **partial emulation of the underlying hardware** is provided. Unlike full virtualization, it **does not allow complete isolation** or full execution of the guest operating system.

In partial virtualization, only some hardware components are virtualized, while others are shared directly with the host system.

Partial virtualization, **or software-based virtualization,** involves virtualizing specific components of the hardware while leaving others to be managed by the host operating system.

Many applications can run transparently, but **not all OS features are fully supported**.

Since the hardware is only partially emulated, certain privileged operations may not function properly.

It offers better performance than full emulation in some cases because fewer resources are virtualized.

A classic example of partial virtualization is **address space virtualization** used in time-sharing systems.

In address space virtualization, multiple users and applications run simultaneously in separate memory spaces.

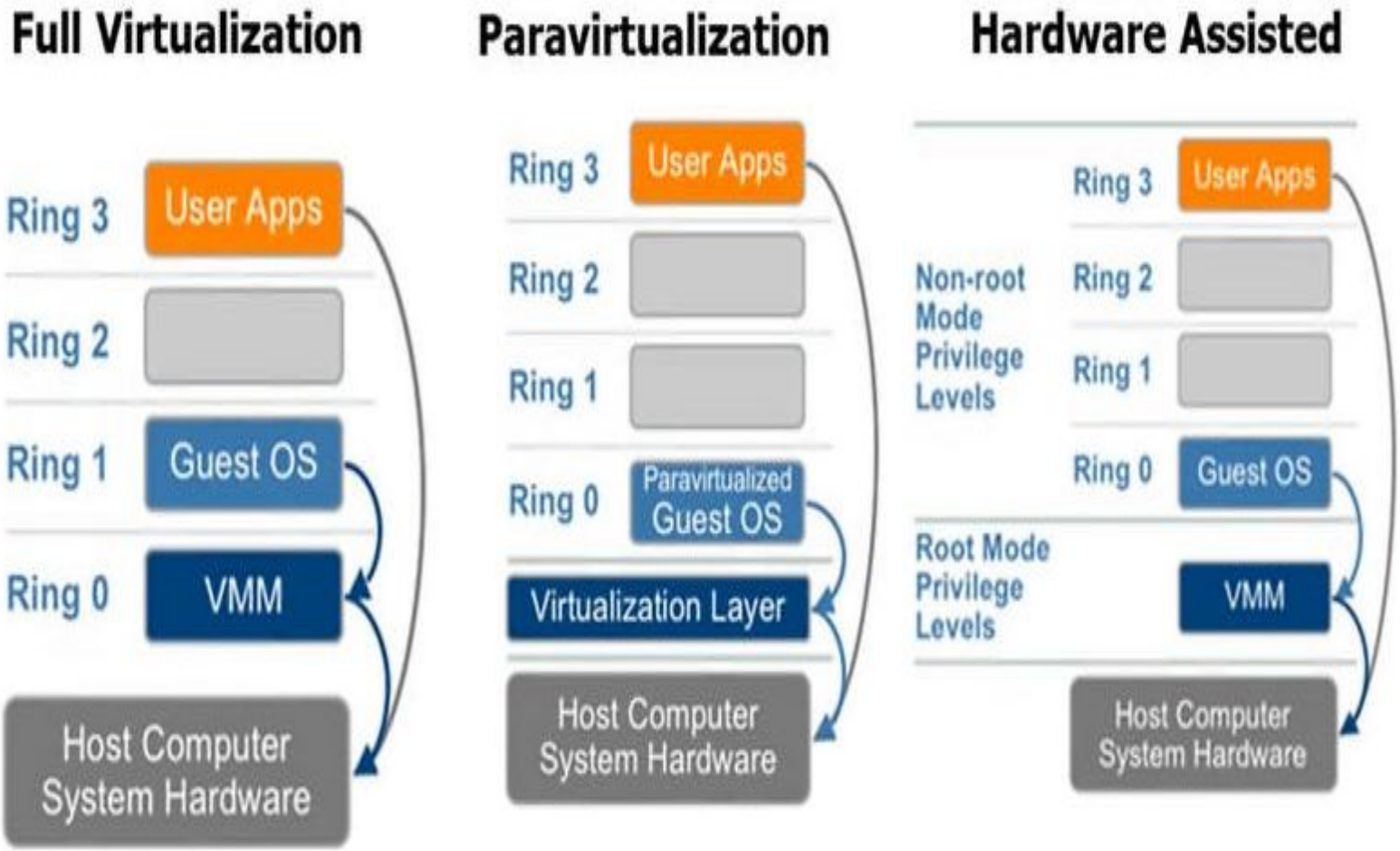
However, they still share common hardware resources such as the **CPU, disk, and network**.

It doesn't completely simulate the underlying hardware.

It was implemented in experimental systems such as the **IBM M44/44X**, and today, address space virtualization is a standard feature in modern operating systems.

Full virtualization vs. paravirtualization

Full virtualization	Paravirtualization
The VM enables execution of instructions from unmodified OSes	The VM uses an API to port an OS to the hypervisor
OSes require no modification	OSes require modifications
Provides complete logical isolation	Not fully isolated; poses security risks in the API
Highly portable and compatible	Less portable and compatible
Uses binary translation and direct calls	Uses hypercalls through an API


Full Virtualization
Ring 3
User Apps
Ring 2
Ring 1
Guest OS
Ring 0
VMM
Host Computer System Hardware
Paravirtualization
Ring 3
User Apps
Ring 2
Ring 1
Ring 0
Paravirtualized Guest OS
Virtualization Layer
Host Computer System Hardware
Hardware Assisted
Non-root Mode Privilege Levels
Ring 3
User Apps
Ring 2
Ring 1
Ring 0
Guest OS
Root Mode Privilege Levels
VMM
Host Computer System Hardware

4. Operating system-level virtualization

Operating system-level virtualization offers the opportunity to create different and separated execution environments for applications that are managed concurrently. Differently from hardware virtualization, there is **no virtual machine manager or hypervisor**, and the virtualization is done within a single operating system, where the OS kernel allows for multiple isolated user space instances. The kernel is also responsible for sharing the system resources among instances and for limiting the impact of instances on each other. A user space instance in general contains a proper view of the file system, which is completely isolated, and separate IP addresses, software configurations, and access to devices. Operating systems supporting this type of virtualization are general-purpose, timeshared operating systems with the capability to provide stronger namespace and resource isolation.

This virtualization technique can be considered an evolution of the chroot mechanism in Unix systems. The chroot operation changes the file system root directory for a process and its children to a specific directory. As a result, the process and its children cannot have access to other portions of the file system than those accessible under the new root directory. Because Unix systems also expose devices as parts of the file system, by using this method it is possible to completely isolate a set of processes.

This technique is an efficient solution for server consolidation scenarios in which multiple application servers share the same technology: operating system, application server framework, and other components. When different servers are aggregated into one physical server, each server is run in a different user space, completely isolated from the others.

Examples of operating system-level virtualizations are FreeBSD Jails, IBM Logical Partition (LPAR), SolarisZones and Containers, Parallels Virtuozzo Containers, OpenVZ, iCore Virtual Accounts, Free Virtual Private Server (FreeVPS), and others.

5. Programming language-level virtualization

Programming language-level virtualization is mostly used to achieve ease of deployment of applications, managed execution, and portability across different platforms and operating systems. It consists of a virtual machine executing the byte code of a program, which is the result of the compilation process. Compilers implemented and used this technology to produce a binary format representing the machine code for an abstract architecture. The characteristics of this architecture vary from implementation to implementation. Generally these virtual machines constitute a simplification of the underlying hardware instruction set and provide some high-level instructions that map some of the features of the languages compiled for them. At runtime, the byte code can be either interpreted or compiled on the fly—or jitted —against the underlying hardware instruction set.

Programming language-level virtualization has a long trail in computer science history and originally was used in 1966 for the implementation of Basic Combined Programming Language (BCPL), a language for writing compilers and one of the ancestors of the C programming language.

The ability to support multiple programming languages has been one of the key elements of the Common Language Infrastructure (CLI), which is the specification behind.NET Framework. Currently, the Java platform and .NET Framework represent the most popular technologies for enterprise application development. Both Java and the CLI are stack-based virtual machines: The reference model of the abstract architecture is based on an execution stack that is used to perform operations. The byte code generated by compilers for these architectures contains a set of instructions that load operands on the stack, perform some operations with them, and put the result on the stack.

The main advantage of programming-level virtual machines, also called process virtual machines, is the ability to provide a uniform execution environment across different platforms. Programs compiled into byte code can be executed on any operating system and platform for which a virtual machine able to execute that code has been provided.

6.Application-level virtualization

Application-level virtualization is a technique allowing applications to be run in runtime environments that do not natively support all the features required by such applications. In this scenario, applications are not installed in the expected runtime environment but are run as though they were. In general, these techniques are mostly concerned with partial file systems, libraries, and operating system component emulation. Such emulation is performed by a thin layer—a program or an operating system component—that is in charge of executing the application.

Emulation can also be used to execute program binaries compiled for different hardware architectures. In this case, one of the following strategies can be implemented:

• **Interpretation**. In this technique every source instruction is interpreted by an emulator for executing native ISA instructions, leading to poor performance. Interpretation has a minimal startup cost but a huge overhead, since each instruction is emulated.

• **Binary translation.** In this technique every source instruction is converted to native instructions with equivalent functions. After a block of instructions is translated, it is cached and reused. Binary translation has a large initial overhead cost, but over time it is subject to better

performance, since previously translated instruction blocks are directly executed.

Emulation, as described, is different from hardware-level virtualization. The former simply allows the execution of a program compiled against a different hardware, whereas the latter emulates a complete hardware environment where an entire operating system can be installed.

Application virtualization is a good solution in the case of missing libraries in the host operating system; in this case a replacement library can be linked with the application, or library calls can be remapped to existing functions available in the host system. Another advantage is that in this case the virtual machine manager is much lighter since it provides a partial emulation of the runtime environment compared to hardware virtualization.

Other types of virtualization:

Storage virtualization:

Storage virtualization is a system administration practice that allows decoupling the physical organization of the hardware from its logical representation. Using this technique, users do not have to be worried about the specific location of their data, which can be identified using a logical path.

Storage virtualization allows us to harness a wide range of storage facilities and represent them under a single logical file system.

Network virtualization

Network virtualization combines hardware appliances and specific software for the creation and management of a virtual network. Network virtualization can aggregate different physical networks into a single logical network (external network virtualization) or provide network-like functionality to an operating system partition (internal network virtualization).

The result of external network virtualization is generally a virtual LAN (VLAN). A VLAN is an aggregation of hosts that communicate with each other as though they were located under the same broadcasting domain. Internal network virtualization is generally applied together with hardware and operating system-level virtualization, in which the guests obtain a virtual network interface to communicate with. There are several options for implementing internal network virtualization: The guest can share the same network interface of the host and use Network Address Translation (NAT) to access the network; the virtual machine manager can emulate, and install on the host, an additional network device, together with the driver; or the guest can have a private network only with the guest.

Desktop virtualization

- Desktop virtualization abstracts the desktop environment available on a personal computer in order to provide access to it using a client/server approach. Desktop virtualization provides the same outcome of hardware virtualization but serves a different purpose. Similarly to hardware virtualization, desktop virtualization makes accessible a different system as though it were natively installed on the host, but this system is remotely stored on a different host and accessed through a network connection. Moreover, desktop virtualization addresses the problem of making the same desktop environment accessible from everywhere. Although the term desktop virtualization strictly refers to the ability to remotely access a desktop environment, generally the desktop environment is stored in a remote server or a data center that provides a high-availability infrastructure and ensures the accessibility and persistence of the data.

Application server virtualization

Application server virtualization abstracts a collection of application servers that provide the same services as a single virtual application server by using load-balancing strategies and providing a high-availability infrastructure for the services hosted in the application server. This is a particular form of virtualization and serves the same purpose of storage virtualization: providing a better quality of service rather than emulating a different environment.

Virtualization and cloud computing

Virtualization plays an important role in cloud computing since it allows for the appropriate degree of customization, security, isolation, and manageability that are fundamental for delivering IT services on demand. Virtualization technologies are primarily used to offer configurable computing environments and storage. Network virtualization is less popular and, in most cases, is a complementary feature, which is naturally needed in build virtual computing systems.

virtualization allows us to create isolated and controllable environments, it is possible to serve these environments with the same resource without them interfering with each other. If the underlying resources are capable enough, there will be no evidence of such sharing. This opportunity is particularly attractive when resources are underutilized, because it allows reducing the number of active resources by aggregating virtual machines over a smaller number of resources that become fully utilized. This practice is also known as server consolidation, while the movement of virtual machine instances is called virtual machine migration.

Because virtual machine instances are controllable environments, consolidation can be applied with a minimum impact, either by temporarily stopping its execution and moving its data to the new resources or by performing a finer control and moving the instance while it is running. This second techniques is known as live migration and in general is more complex to implement but more efficient since there is no disruption of the activity of the virtual machine instance.

Server consolidation and virtual machine migration are principally used in the case of hardware virtualization, even though they are also technically possible in the case of programming language virtualization

Storage virtualization constitutes an interesting opportunity given by virtualization technologies, often complementary to the execution of virtualization.

Finally, cloud computing revamps the concept of desktop virtualization, initially introduced in the mainframe era. The ability to recreate the entire computing stack—from infrastructure to application services—on demand opens the path to having a complete virtual computer hosted on the infrastructure of the provider and accessed by a thin client over a capable Internet connection.

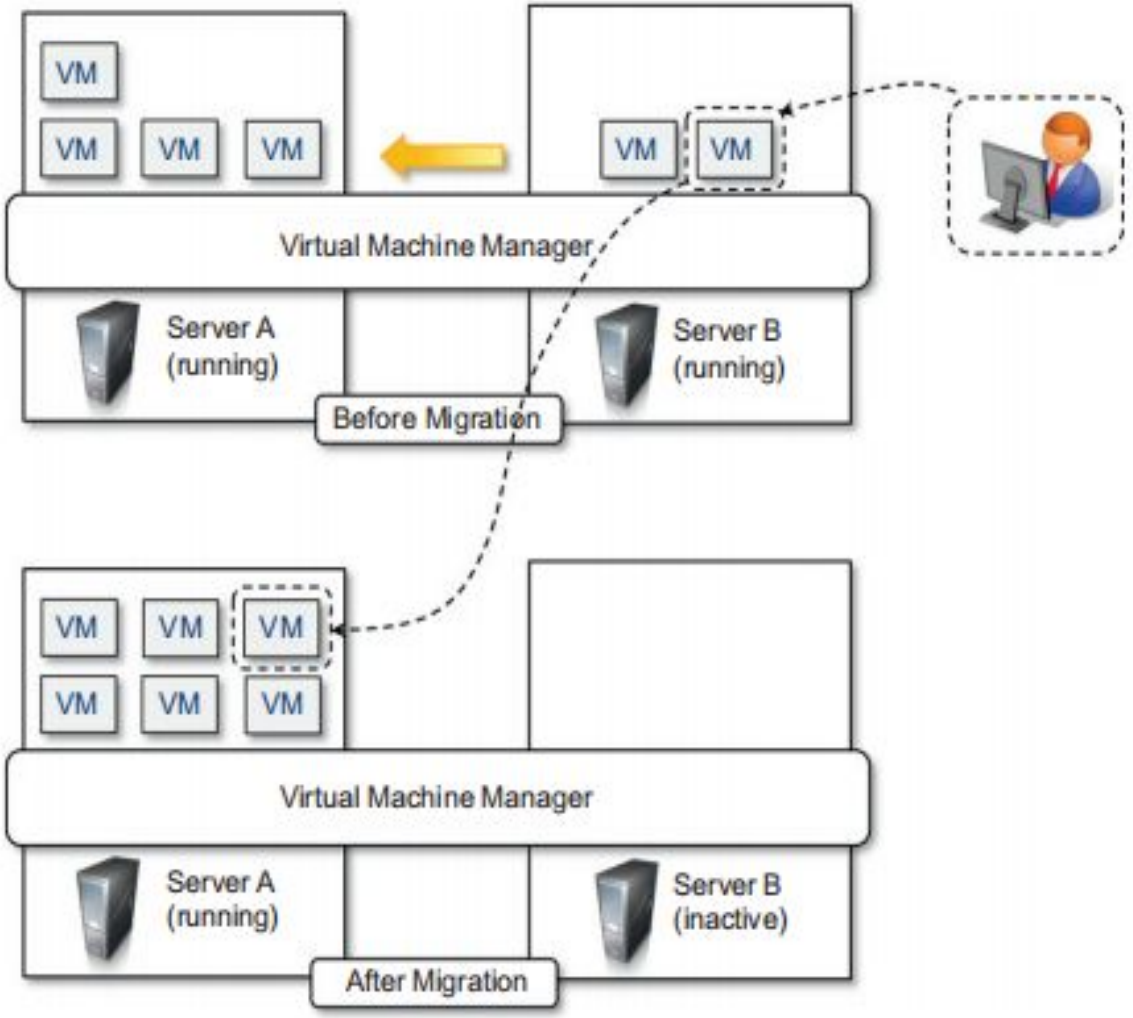


FIGURE 3.10

Live migration and server consolidation.

Pros and cons of virtualization

Virtualization has now become extremely popular and widely used, especially in cloud computing. The primary reason for its wide success is the elimination of technology barriers that prevented virtualization from being an effective and viable solution in the past.

Advantages of virtualization

1. **Managed Execution:** Virtualization enables controlled execution of applications within a virtual environment, ensuring better management of computing processes.
2. **Isolation:** Each virtual machine (VM) operates independently, preventing harmful operations in one VM from affecting others or the host system.
3. **Sandbox Environment:** Virtual environments can be configured as sandboxes, restricting unauthorized or malicious activities from crossing system boundaries.
4. **Efficient Resource Allocation:** Resources such as CPU, memory, and storage can be easily allocated and partitioned among multiple virtual machines through the virtual machine manager.
5. **Improved Quality of Service (QoS):** Fine-tuning and controlled distribution of resources enhance performance management, especially in server consolidation scenarios.
6. **Portability:** Virtual machine instances are typically stored as files, making them easy to move across different physical systems.
7. **Self-Containment:** Virtual machines generally require only a virtual machine manager to run, reducing external dependencies and simplifying administration.
8. **Support for Migration:** Portability enables VM migration across physical hosts, facilitating load balancing and efficient server consolidation.
9. **Enhanced Security and Hardware Protection:** Since applications run in virtual environments, the risk of direct damage to underlying hardware is significantly reduced.
10. **Efficient Resource Utilization and Energy Savings:** Multiple virtual machines can securely share host resources, enabling dynamic adjustment of active physical resources, reducing energy consumption, and minimizing environmental impact.

Disadvantages :

Virtualization also has downsides. The most evident is represented by a performance decrease of guest systems as a result of the intermediation performed by the virtualization layer.

1. **Performance Overhead:** Virtualization introduces an abstraction layer between the guest and host, causing increased latency due to virtual CPU management, privileged instruction handling, memory paging, and I/O operations.
2. **Runtime Slowdowns:** In programming-level virtual machines such as **Java Virtual Machine** and **.NET**, interpretation and binary translation can reduce execution speed, though techniques like native/JIT compilation help improve performance.
3. **Resource Inefficiency:** Some host hardware features (e.g., advanced graphics or device drivers) may not be fully exposed in virtual environments, leading to limited functionality and degraded user experience.
4. **Security Threats:** Virtualization can introduce new attack vectors, including malware like **Blue Pill** and **SubVirt**, which exploit virtualization layers to gain hidden control over systems.
5. **Hardware-Based Improvements:** Modern processor support such as **Intel VT** and **AMD Pacifica** has significantly reduced performance issues and strengthened virtualization security.

Technology examples

A wide range of virtualization technology is available especially for virtualizing computing environments. In this section, we discuss the most relevant technologies and approaches utilized in the field. Cloud-specific solutions are discussed in the next chapter.

1. Xen: paravirtualization

Xen is an open-source initiative implementing a virtualization platform based on paravirtualization. Initially developed by a group of researchers at the University of Cambridge in the United Kingdom, Xen now has a large open-source community backing it. Citrix also offers it as a commercial solution, Xen Source. Xen-based technology is used for either desktop virtualization or server virtualization, and recently it has also been used to provide cloud computing solutions by means of Xen Cloud Platform (XCP). At the basis of all these solutions is the Xen Hypervisor, which constitutes the core technology of Xen. Recently Xen has been advanced to support full virtualization using hardware-assisted virtualization.

Xen is the most popular implementation of paravirtualization, which, in contrast with full virtualization, allows high-performance execution of guest operating systems. This is made possible by eliminating the performance loss while executing instructions that require special management.

This is done by modifying portions of the guest operating systems run by Xen with reference to the execution of such instructions. Therefore it is not a transparent solution for implementing virtualization. This is particularly true for x86, which is the most popular architecture on commodity machines and servers.

Figure describes the architecture of Xen and its mapping onto a classic x86 privilege model. A Xen-based system is managed by the Xen hypervisor, which runs in the highest privileged mode and controls the access of guest operating system to the underlying hardware.

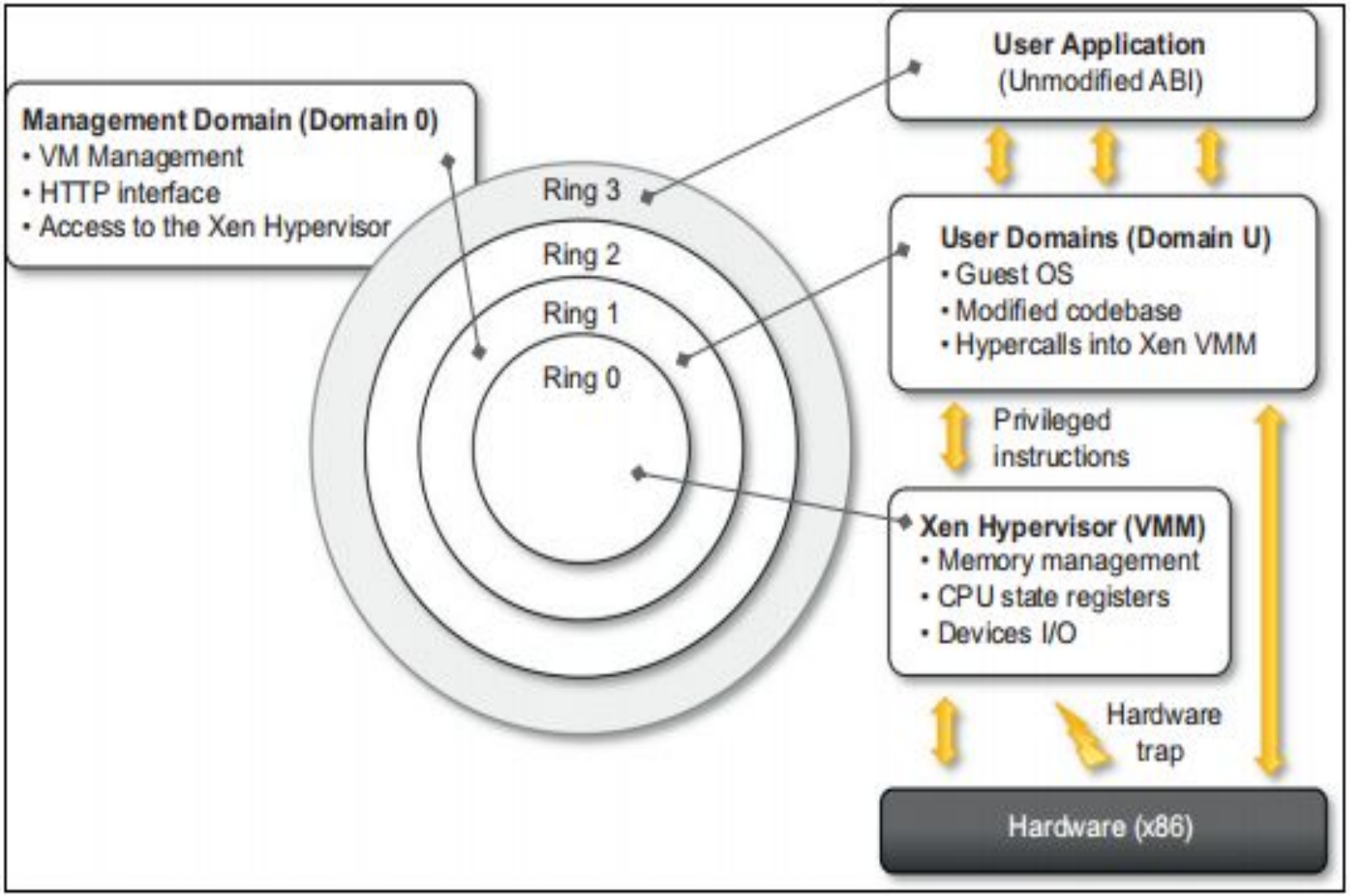


FIGURE 3.11

Xen architecture and guest OS management.

2. VMware: full virtualization

VMware's technology is based on the concept of full virtualization, where the underlying hardware is replicated and made available to the guest operating system, which runs unaware of such abstraction layers and does not need to be modified. VMware implements full virtualization either in the desktop environment, by means of Type II hypervisors, or in the server environment, by means of Type I hypervisors. In both cases, full virtualization is made possible by means of direct execution (for non sensitive instructions) and binary translation (for sensitive instructions), thus allowing the virtualization of architecture such as x86.

Besides these two core solutions, VMware provides additional tools and software that simplify the use of virtualization technology either in a desktop environment, with tools enhancing the integration of virtual guests with the host, or in a server environment, with solutions for building and managing virtual computing infrastructures.

2.1 Full virtualization and binary translation

VMware is well known for the capability to virtualize x86 architectures, which runs unmodified on top of their hypervisors. With the new generation of hardware architectures and the introduction of hardware-assisted virtualization (Intel VT-x and AMD V) in 2006, full virtualization is made possible with hardware support, but before that date, the use of dynamic binary translation was the only solution that allowed running x86 guest operating systems unmodified in a virtualized environment.

CPU virtualization is only a component of a fully virtualized hardware environment. VMware achieves full virtualization by providing virtual representation of memory and I/O devices. Memory virtualization constitutes another challenge of virtualized environments and can deeply impact performance without the appropriate hardware support. The main reason is the presence of a memory management unit (MMU), which needs to be emulated as part of the virtual hardware. Especially in the case of hosted hypervisors (Type II), where the virtual MMU and the host-OS MMU are traversed sequentially before getting to the physical memory page, the impact on performance can be significant.

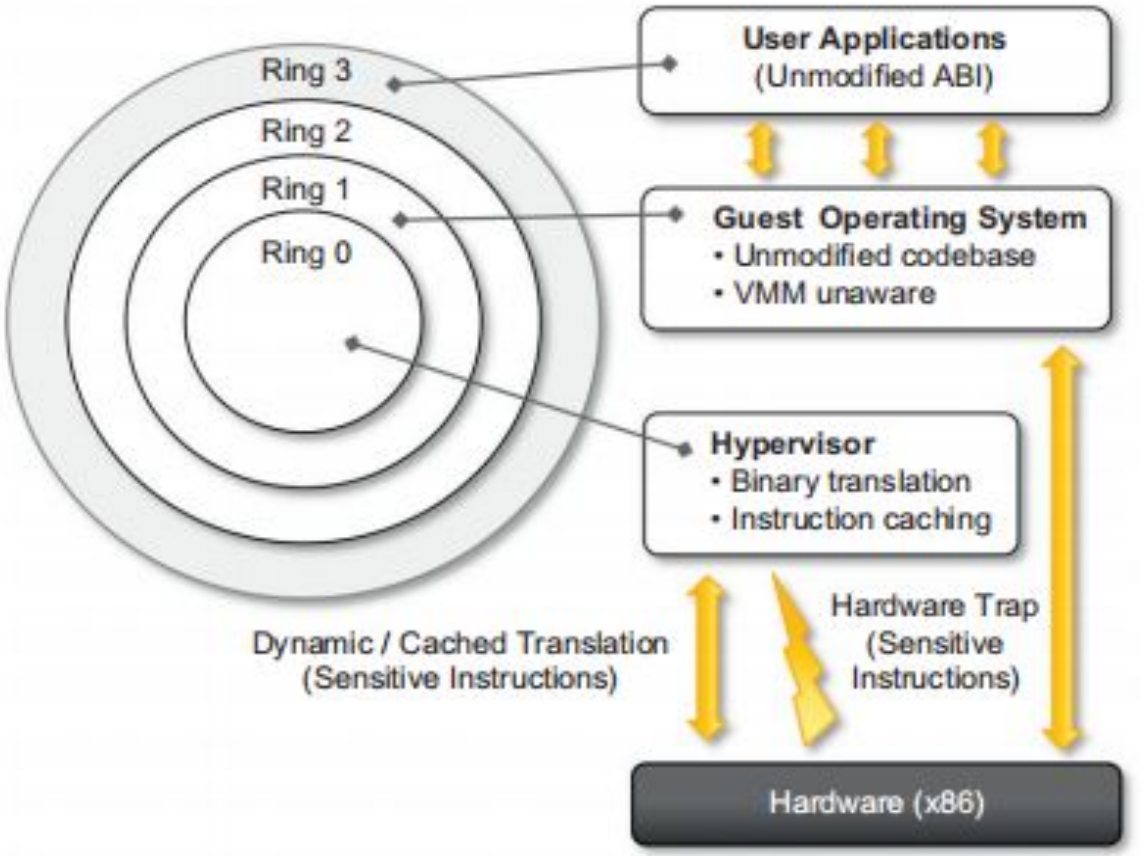


FIGURE 3.12

A full virtualization reference model.

Finally, VMware also provides full virtualization of I/O devices such as network controllers and other peripherals such as keyboard, mouse, disks, and universal serial bus (USB) controllers.

2.2 Virtualization solutions

VMware is a pioneer in virtualization technology and offers a collection of virtualization solutions covering the entire range of the market, from desktop computing to enterprise computing and infrastructure virtualization.

End-user (desktop) virtualization

VMware supports virtualization of operating system environments and single applications on enduser computers. The first option is the most popular and allows installing a different operating systems and applications in a completely isolated environment from the hosting operating system. Specific VMware software—VMware Workstation, for Windows operating systems, and VMware Fusion, for Mac OS X environments—is installed in the host operating system to create virtual machines and manage their execution.

The virtualization environment is created by an application installed in guest operating systems, which provides those operating systems with full hardware virtualization of the underlying hardware. This is done by installing a specific driver in the host operating system that provides two main services:

- It deploys a virtual machine manager that can run in privileged mode.
- It provides hooks for the VMware application to process specific I/O requests eventually by relaying such requests to the host operating system via system calls.

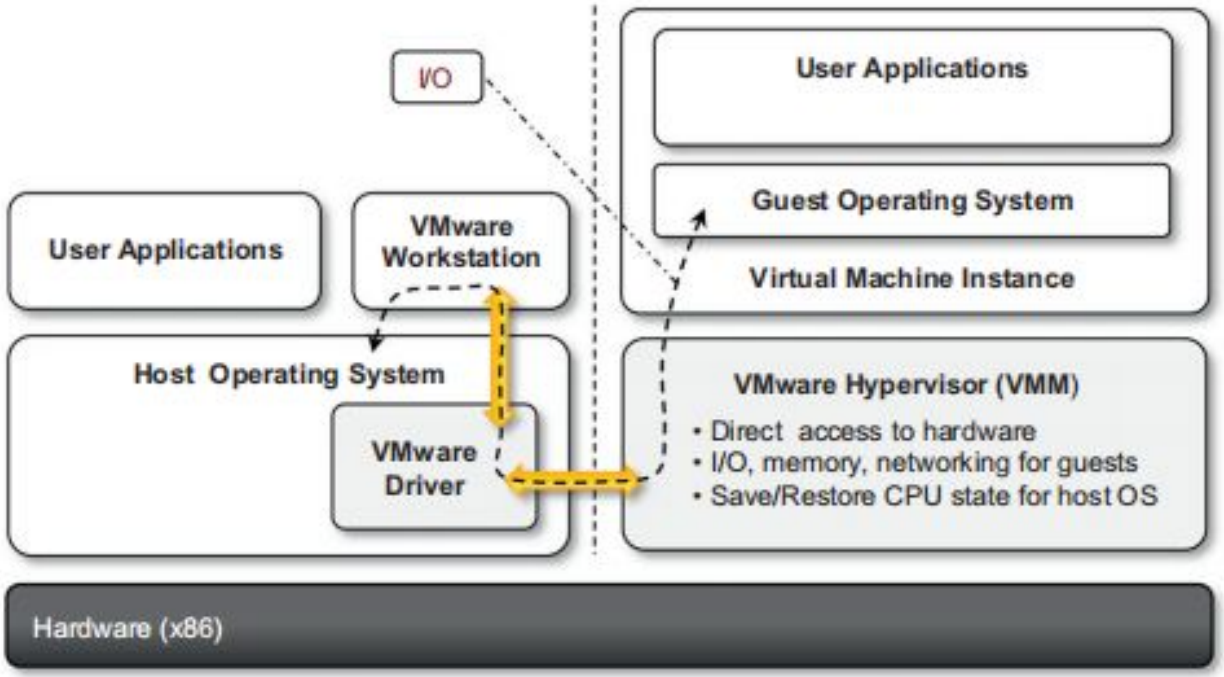


FIGURE 3.13

VMware workstation architecture.

Server virtualization

VMware provided solutions for server virtualization with different approaches over time. Initial support for server virtualization was provided by VMware GSX server, which replicates the approach used for end-user computers and introduces remote management and scripting capabilities. The architecture of VMware GSX Server is depicted in Figure

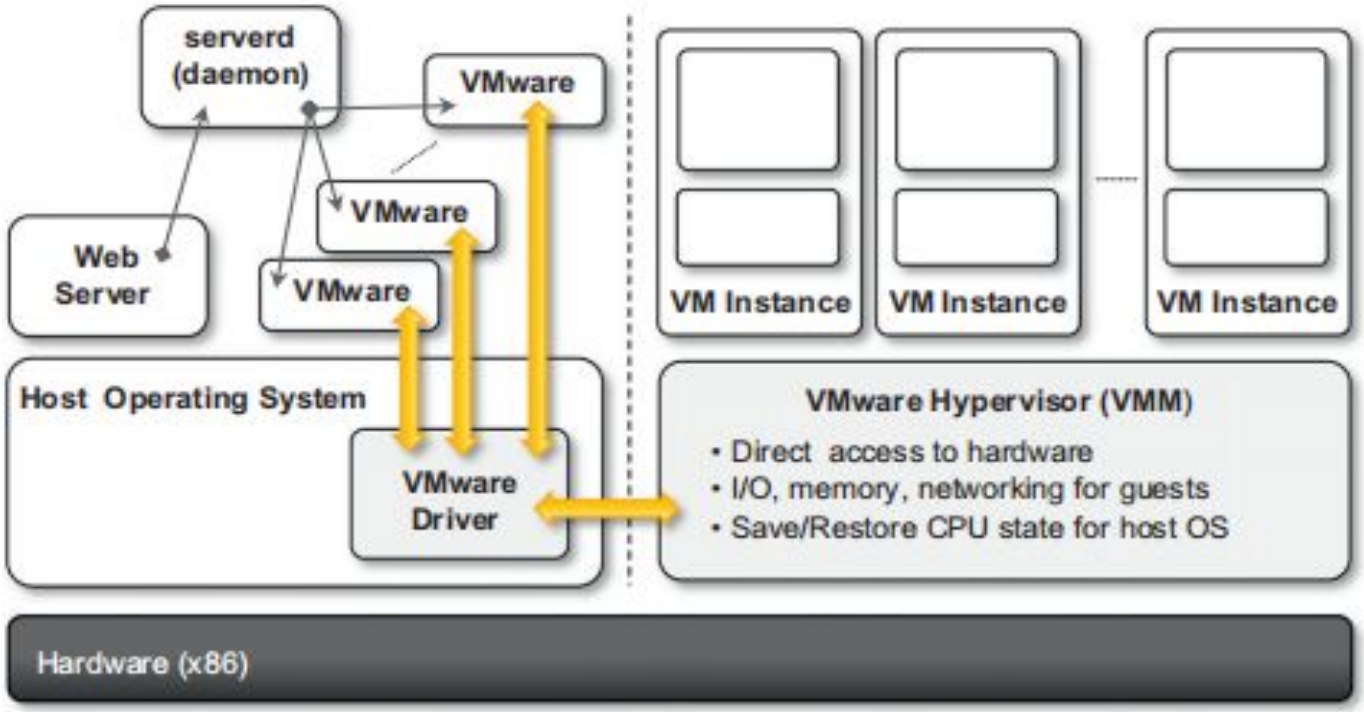


FIGURE 3.14

VMware GSX server architecture.

3 Microsoft Hyper-V

Hyper-V is an infrastructure virtualization solution developed by Microsoft for server virtualization. As the name recalls, it uses a hypervisor-based approach to hardware virtualization, which leverages several techniques to support a variety of guest operating systems. Hyper-V is currently shipped as a component of Windows Server 2008 R2 that installs the hypervisor as a role within the server.

3.1 Architecture

Hyper-V supports multiple and concurrent execution of guest operating systems by means of partitions. A partition is a completely isolated environment in which an operating system is installed and run.

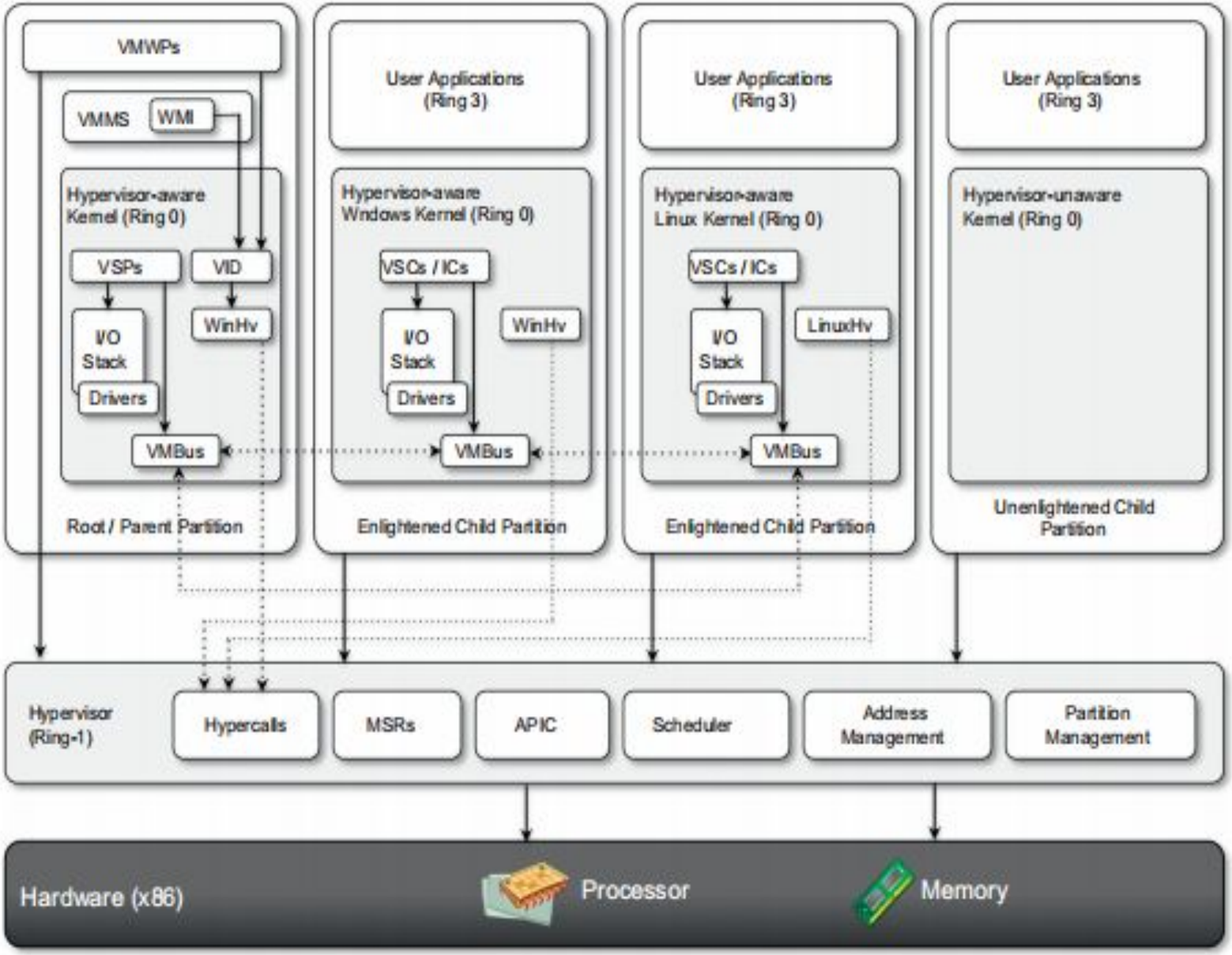


FIGURE 3.17

Microsoft Hyper-V architecture.